

DIGITAL ELECTRONICS

ET 8301

Monday, January 13, 2025

Mr. Cuthbert John Karawa

Lecture 09

COUNTER

College of Information and Communication Technology (CoICT)

Department Of Electronics And Telecommunications Engineering.

Introduction

- The flipflop are essential component in clocked sequential circuits.
- Circuits that include flip-flops are usually classified by the functions they perform.
- **Counters** and **registers** belong to the category of MSI sequential logic circuits.
- They have **similar architecture**, as both counters and registers **comprise a cascaded arrangement** of more than one **flip-flop** with or without combinational logic devices.
- Counters are mainly used in **counting applications**.

Introduction

- They can either measure the time interval between two unknown time instants or measure the frequency of a given signal.
- The main purpose of counter is to **count** the number of occurrence of input.
- The number of flip-flops used and the way in which they are **connected** determine the **number of states** (called the **modulus**) and the specific sequence of states that the counter goes through during each complete cycle.
- For **N-Bit** counter will have **N flipflop** and **2^N States**.

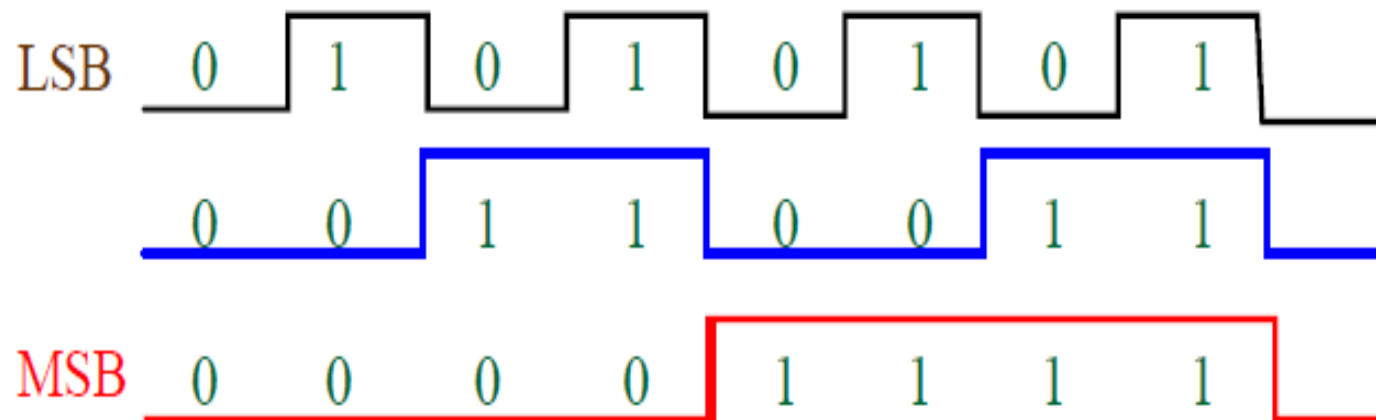
Introduction

- Counting circuits can be in either synchronous (**clock driven**) and asynchronous (**event driven**).
- Counters are **classified** into **two** broad categories according to the way they are **clocked**:
 - ✓ **Asynchronous** (ripple) counters
 - ✓ **Synchronous** (parallel) counters.
- All counter circuits count clock pulses and store the number received in an array of memory elements.
- Counters is used for timing, control or sequencing operations.

Introduction

Counting in Binary

- A counter can form the same pattern of **0's** and **1's** with logic levels.
- The first stage in the counter represents the **least significant bit** – notice that these waveforms follow the same pattern as counting in binary.

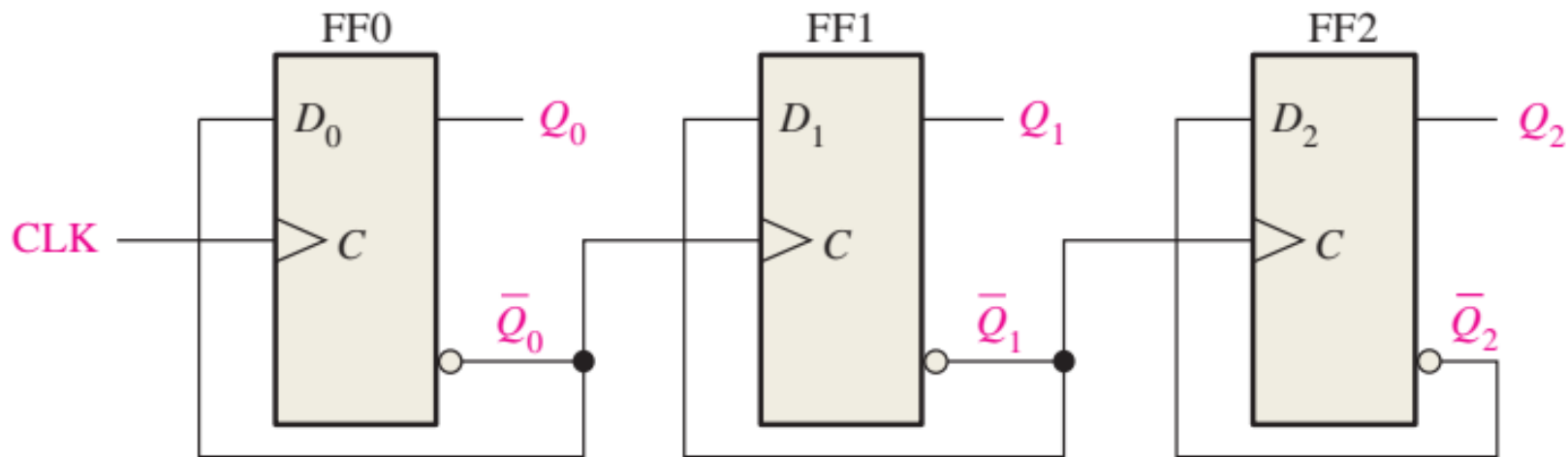


Asynchronous(Ripple) Counters

- In **asynchronous counters**, commonly called **ripple counters**, the first flip-flop is clocked by the external clock pulse and then each successive flip-flop is clocked by the output of the preceding flip-flop.
- Asynchronous refers to events that **do not have a fixed time relationship with each other**, do not occur at the same time.

Asynchronous(Ripple) Counters

- An asynchronous counter is one in which the flip-flops (FF) within the counter **do not change states at exactly the same time** because they **do not have a common clock pulse**.



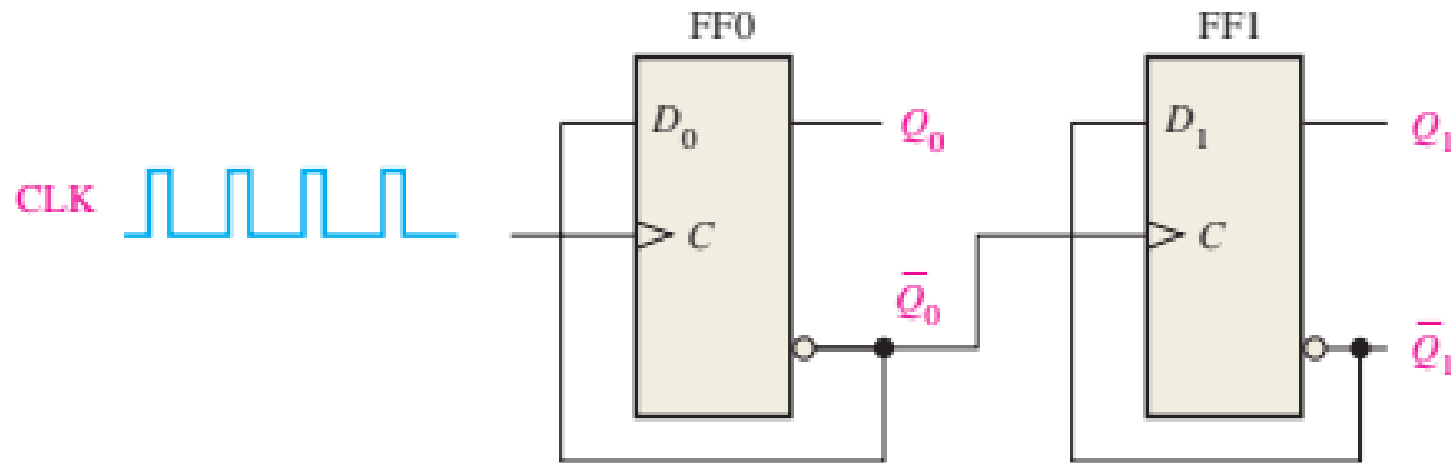
Asynchronous(Ripple) Counters

- A **ripple counter** is a cascaded arrangement of flip-flops where the output of one flip-flop drives the clock input of the following flip-flop.
- The **number of flip-flops** in the cascaded arrangement depends upon the **number of different logic states** that it goes through before it repeats the sequence, a parameter known as the **modulus of the counter**.
- The **clock input is applied only to the first flip-flop** in the cascaded arrangement. The clock input to any subsequent flip-flop comes from the **output of its immediately preceding flip-flop**.

Asynchronous(Ripple) Counters

Two bit Asynchronous Counter

- The two-bit asynchronous counter as shown in figure below.



- Notice that the **clock (CLK)** is applied to the clock input (C) of only the first flip-flop, FF0, which is always **the least significant bit (LSB)**.

Asynchronous(Ripple) Counters

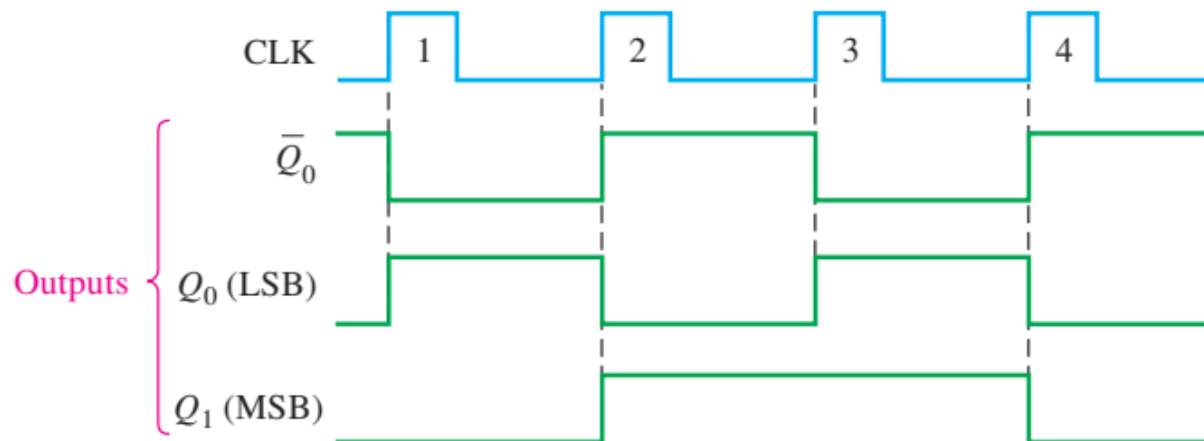
Two bit Asynchronous Counter

- Both flip-flops are connected for toggle operation ($D = \bar{Q}$).
- Assumed to be initially **RESET (Q LOW)**.
- The **positive-going edge of CLK1** (clock pulse 1) causes the Q_0 output of FF0 to go **HIGH**, as shown in Figure on the next slide.
- At the same time the $\bar{Q}_0$ output goes **LOW**, but it has **no effect on FF1** because a positive-going transition must occur to trigger the flip-flop.
- After the **leading edge of CLK1**, $Q_0 = 1$ and $Q_1 = 0$. The positive-going edge of CLK2 causes **Q_0 to go LOW**.
- Output $\bar{Q}_0$ goes **HIGH and triggers FF1**, causing **Q_1 to go HIGH**.
- After the leading edge of CLK2, $Q_0 = 0$ and **$Q_1 = 1$** . The positive-going edge of CLK3 causes Q_0 to go HIGH again. Output Q_0 goes LOW and has no effect on FF1.

Asynchronous(Ripple) Counters

Two bit Asynchronous Counter

- Notice that the Q_0 output is triggered on the **leading edge** of the clock signal. The following stage is triggered from $\bar{Q}_0$.
- The **leading edge** of $\bar{Q}_0$ is equivalent to the **trailing edge** of Q_0 . The resulting sequence is that of an 2-bit binary up counter.



Asynchronous(Ripple) Counters

Two bit Asynchronous Counter

- The **two bit** counter exhibit **four different states** ($2^2 = 4$).
- If **Q_0** represents the **LSB** and **Q_1** represents the **MSB**, then the sequence of the counter states represents a sequence of binary numbers.

Clock Pulse	Q_1	Q_0
Initially	0	0
1	0	1
2	1	0
3	1	1
4 (recycles)	0	0

Asynchronous(Ripple) Counters

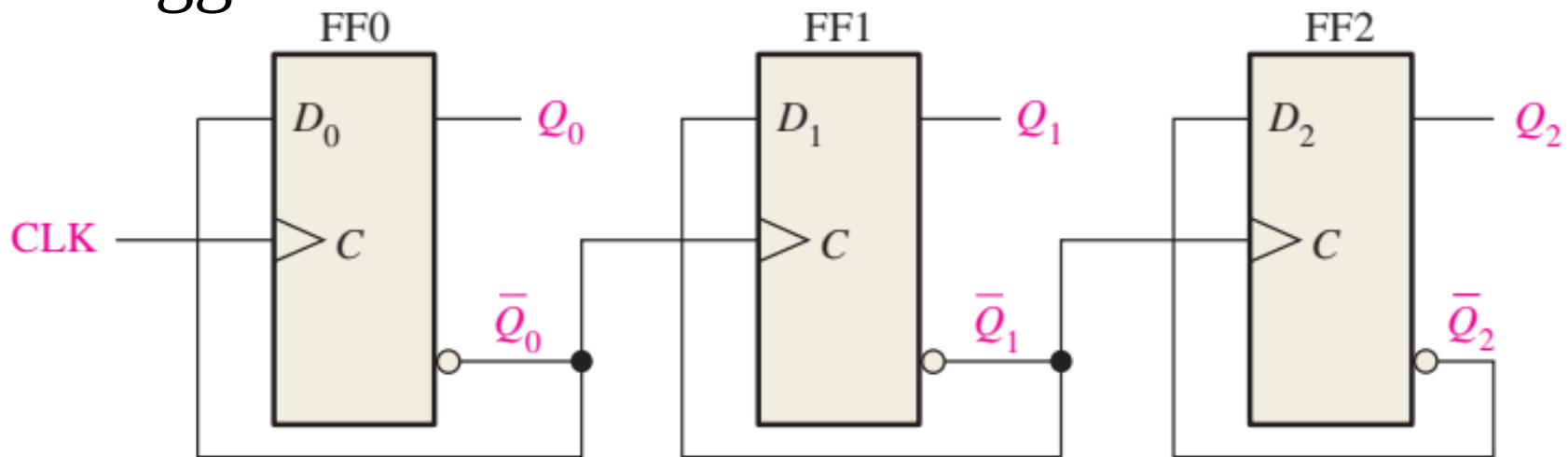
Two bit Asynchronous Counter

- Since it goes through a **binary sequence**, the counter is a **binary counter**.
- It actually counts the number of clock pulses up to three, and on the fourth pulse it **recycles** to its original state (**$Q_0 = 0$, $Q_1 = 0$**).
- The term **recycle** is refers to the transition of the counter from **its final state back** to **its original state**.

Asynchronous(Ripple) Counters

Three bit Asynchronous Counter

- The three-bit asynchronous counter uses three D flip-flops in the toggle mode.

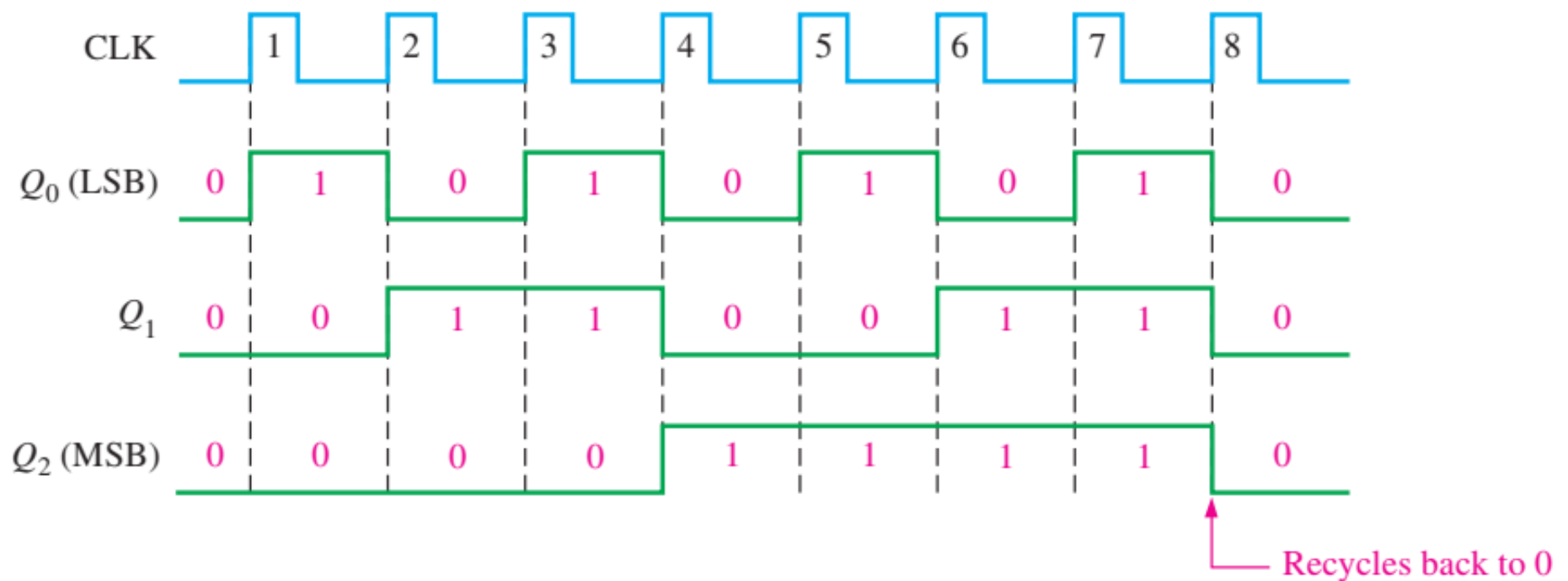


- The basic operation is the same as that of the 2-bit counter except that the **3-bit counter has eight states**, due to its three flip-flops

Asynchronous(Ripple) Counters

Three bit Asynchronous Counter

➤ A timing diagram is shown in Figure below for eight clock pulses.



Asynchronous(Ripple) Counters

Three bit Asynchronous Counter

- **Notice** that the counter progresses through a binary count of **zero** through **seven** and then **recycles** to the **zero state**.
- This counter can be easily expanded for higher count, by connecting additional toggle flip-flops.

Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

Asynchronous(Ripple) Counters

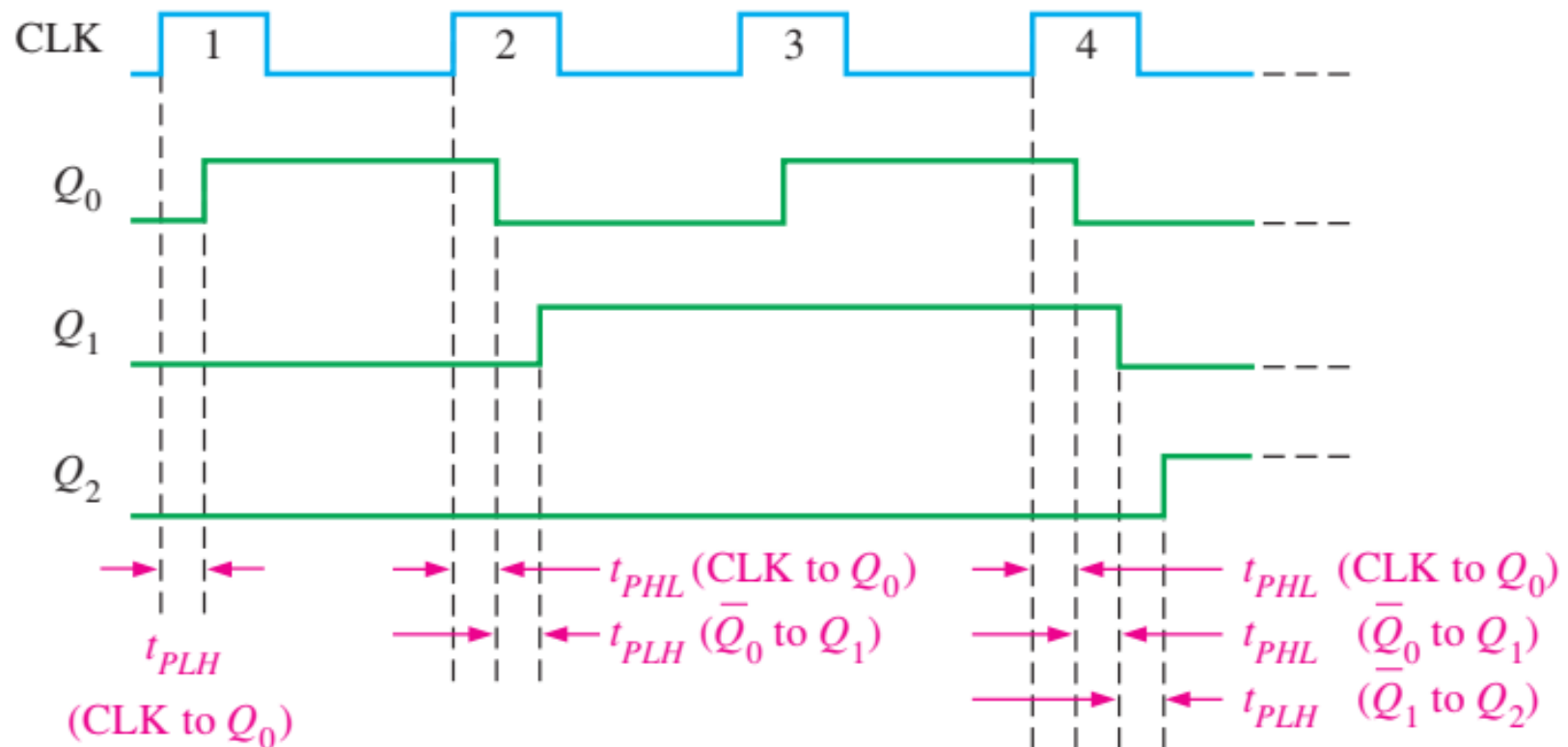
Propagation Delay

- Asynchronous counters are commonly referred to as **ripple counters** for the following reason:
 - ✓ The effect of the input clock pulse is first “felt” by FF0. This effect cannot get to **FF1** immediately because of the propagation delay through **FF0**.
 - ✓ Then there is the propagation delay through FF1 before FF2 can be triggered. Thus, the effect of an input clock pulse “**ripples**” through the counter, taking some time, due to propagation delays, to reach the last flip-flop.

Asynchronous(Ripple) Counters

Propagation Delay

➤ Figure below illustrate the propagation delay in 3-bit asynchronous counter.



Asynchronous(Ripple) Counters

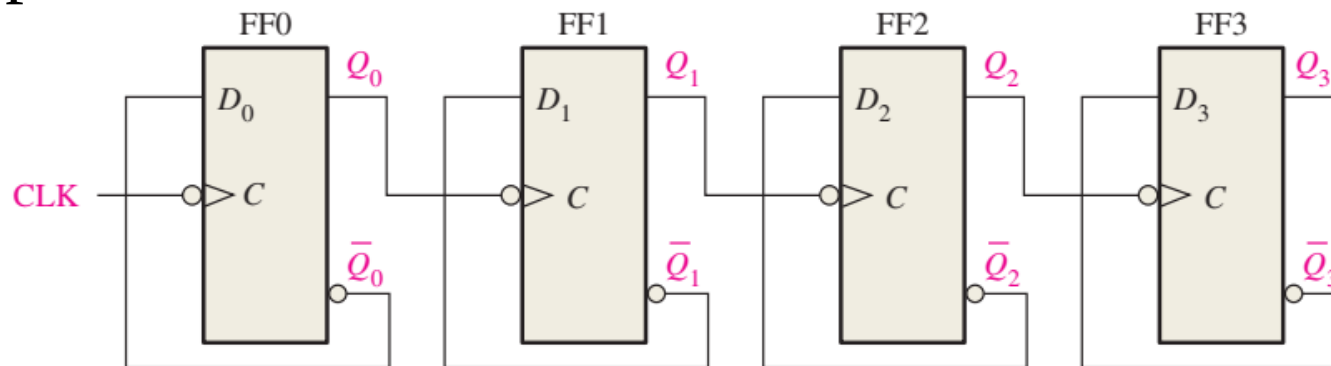
Propagation Delay

- The cumulative delay of an asynchronous counter is a **major disadvantages** because it limits the rate at which the counter can be clocked and creates decoding problems.
- The **maximum cumulative delay** in a counter must be less than the period of the **clock waveform**.

Asynchronous(Ripple) Counters

Example

- A 4-bit asynchronous binary counter is shown in Figure below. Each D flip-flop is negative edge-triggered and has a propagation delay for 10 nanoseconds (ns). Develop a timing diagram showing the Q output of each flip-flop, and determine the total propagation delay time from the triggering edge of a clock pulse until a corresponding change can occur in the state of Q3. Also determine the maximum clock frequency at which the counter can be operated.



Asynchronous Decade Counters

- The **modulus of a counter** is the number of **unique states** through which the counter will sequence.
- The maximum possible number of states (**maximum modulus**) of a counter is 2^n , where **n** is the **number of flip-flops** in the counter.
- Counters can be designed to have a number of states in their sequence that is **less** than the maximum of 2^n . This type of sequence is called a **truncated sequence**.
- One common modulus for counters with truncated sequences is **ten** (called **MOD10**)

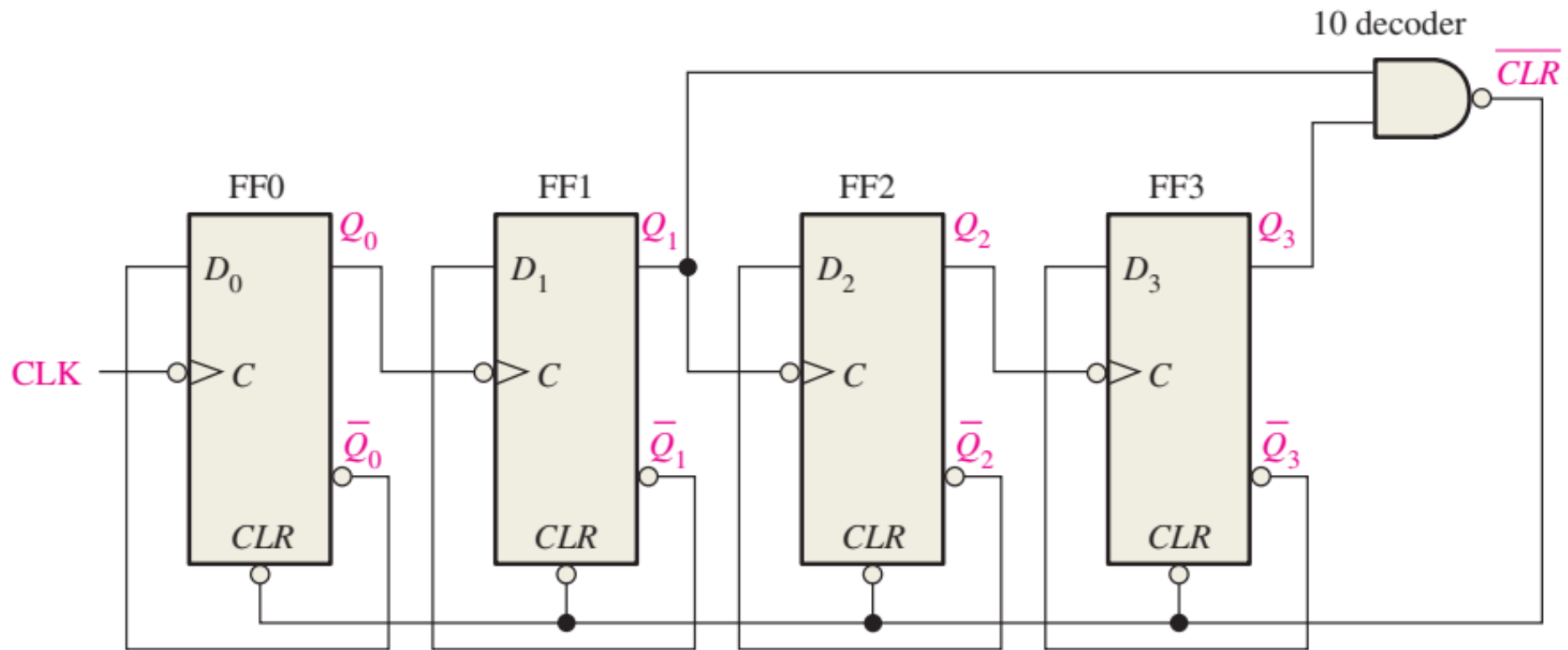
Asynchronous Decade Counters

- Counters with **ten states** in their sequence are called **decade counters**.
- A decade counter with a count sequence of zero (0000) through nine (1001) is a **BCD decade counter** because its ten-state sequence produces the BCD code.
- This type of counter is useful in display applications in which BCD is required for conversion to a decimal readout.
- To obtain a **truncated sequence**, it is necessary **to force the counter to recycle** before going through all of its possible states.

Asynchronous Decade Counters

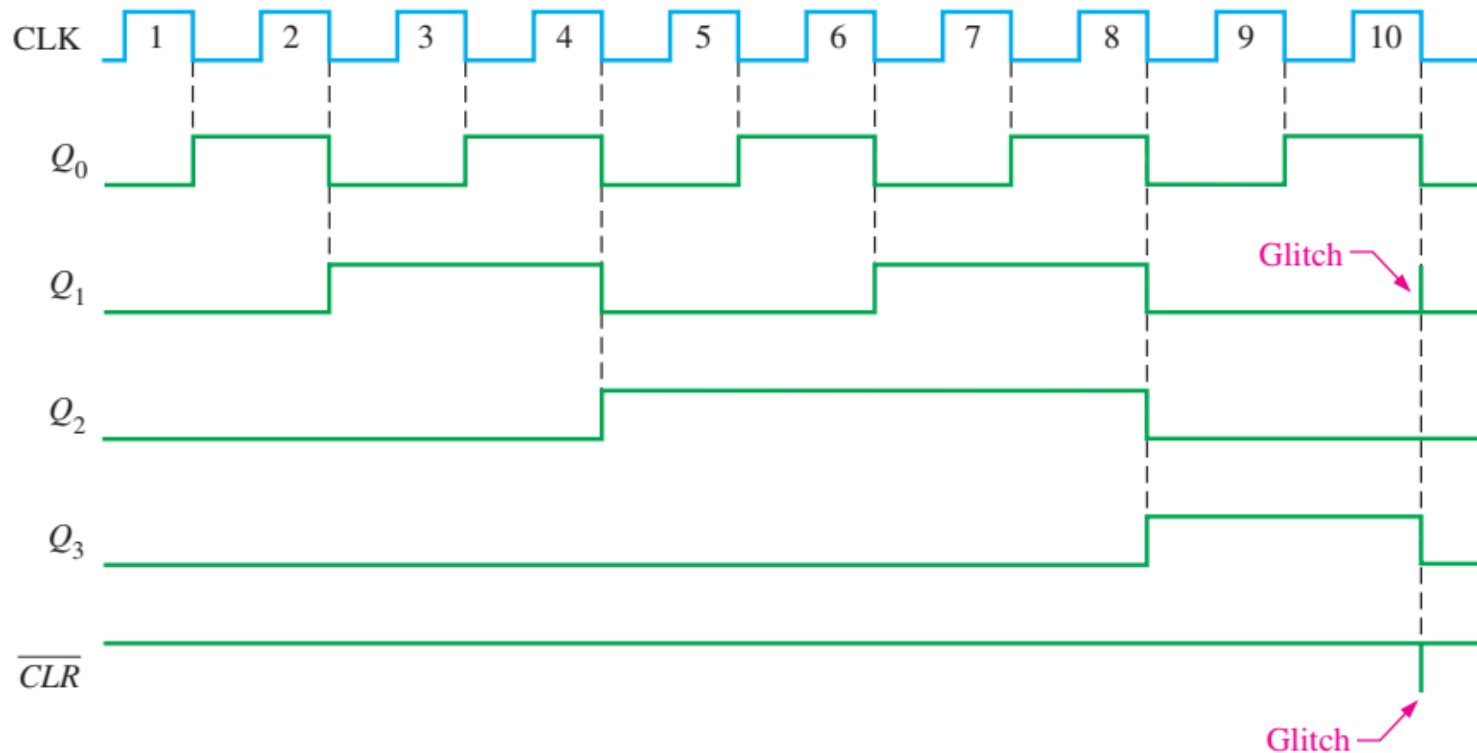
- For **example**, the BCD decade counter must recycle back to the 0000 state after the 1001 state. A decade counter requires **four flip-flops** (three flip-flops are insufficient because $2^3 = 8$).
- One way to make the counter recycle after the count of nine (1001) is to **decode count ten** (1010) with a NAND gate and connect the output of the **NAND** gate to the **clear (CLR)** inputs of the flip-flops, as shown in Figure on the next side.

Asynchronous Decade Counter



Asynchronous Decade Counters

- When the counter goes into **count ten (1010)**, the decoding gate output goes **LOW** and asynchronously **resets all the flip-flops**. The resulting timing diagram is shown in below



Asynchronous Decade Counters

- **Notice** that there is a **glitch** on the Q_1 waveform. The reason for this glitch is that Q_1 must first go **HIGH** before the count of ten can be decoded.
- The counter is in the 1010 state for a short time before it is reset to 0000, thus producing the glitch on Q_1 and the resulting glitch on the CLR line that **resets the counter**.

Synchronous (Parallel) Counters

- The term **synchronous** refers to events that have a **fixed time** relationship with each other.
- A **synchronous counter** is one in which **all the flip-flops** in the **counter are clocked at the same time** by a common clock pulse.
- J-K flip-flops are used to illustrate most synchronous counters.
- D flip-flops can also be used but generally require more logic because of having no direct toggle or no-change states.

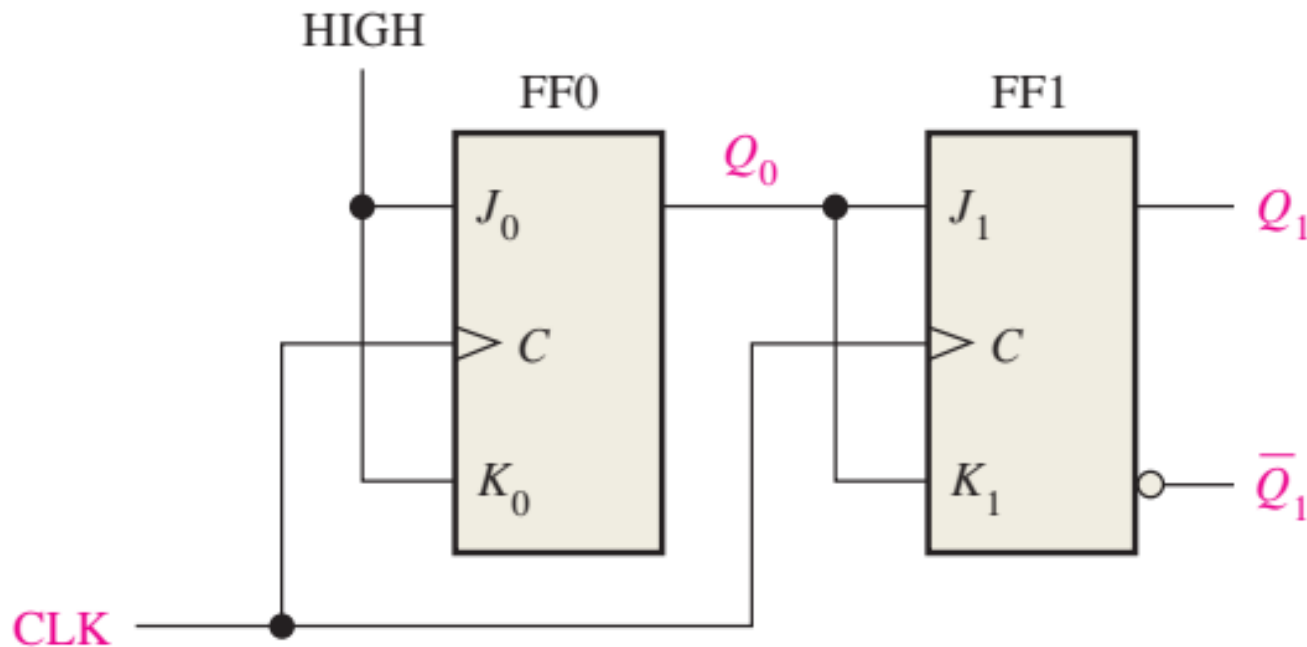
Synchronous(Parallel) Counters

- In synchronous counters, all flip-flops are **clocked together** with **a common clock pulse**.
- Thus, all the flip-flops change state simultaneously.
- The **effect of propagation delay is not cumulative** since all flip-flops in the counter change state **at the same time**.
- Delay if any is due to the delay of a single flip-flop.

Synchronous(Parallel) Counters

Two bit Synchronous Counter

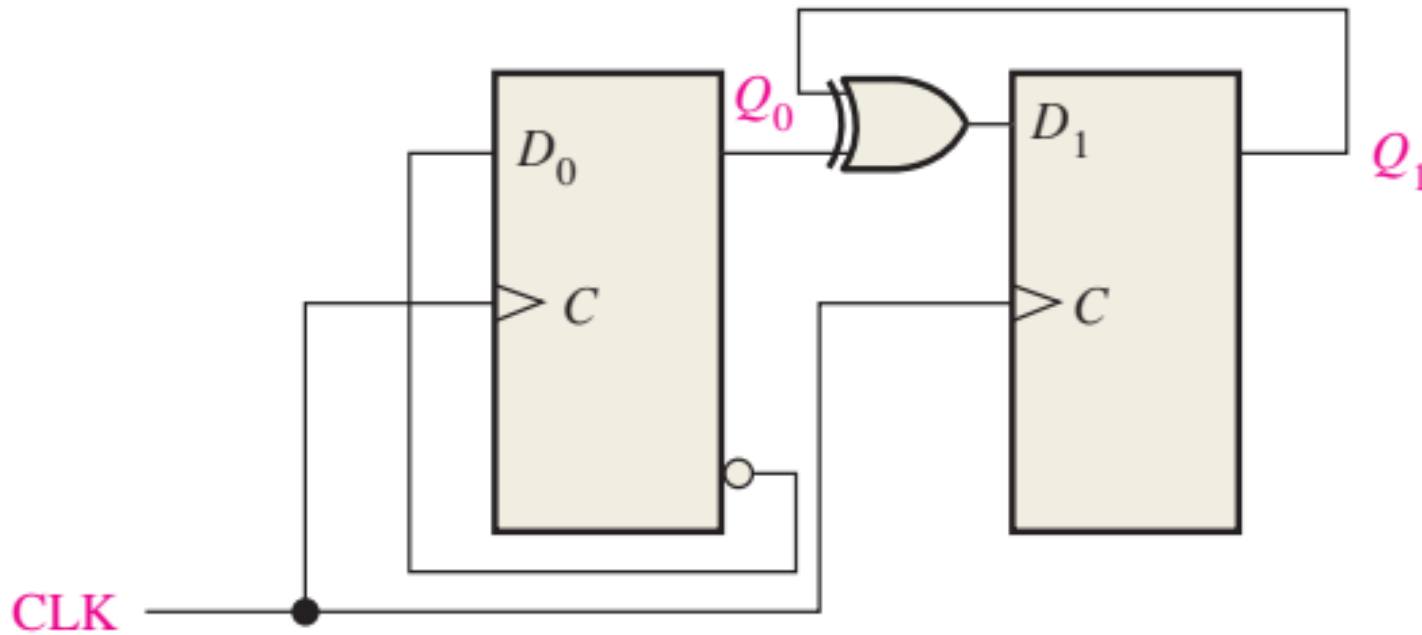
- A two bit synchronous counter consists of two J-K flip-flops as shown in figure.



Synchronous(Parallel) Counters

Two bit Synchronous Counter

- A two bit synchronous counter consists of two D flip-flop implementation is shown below



Synchronous(Parallel) Counters

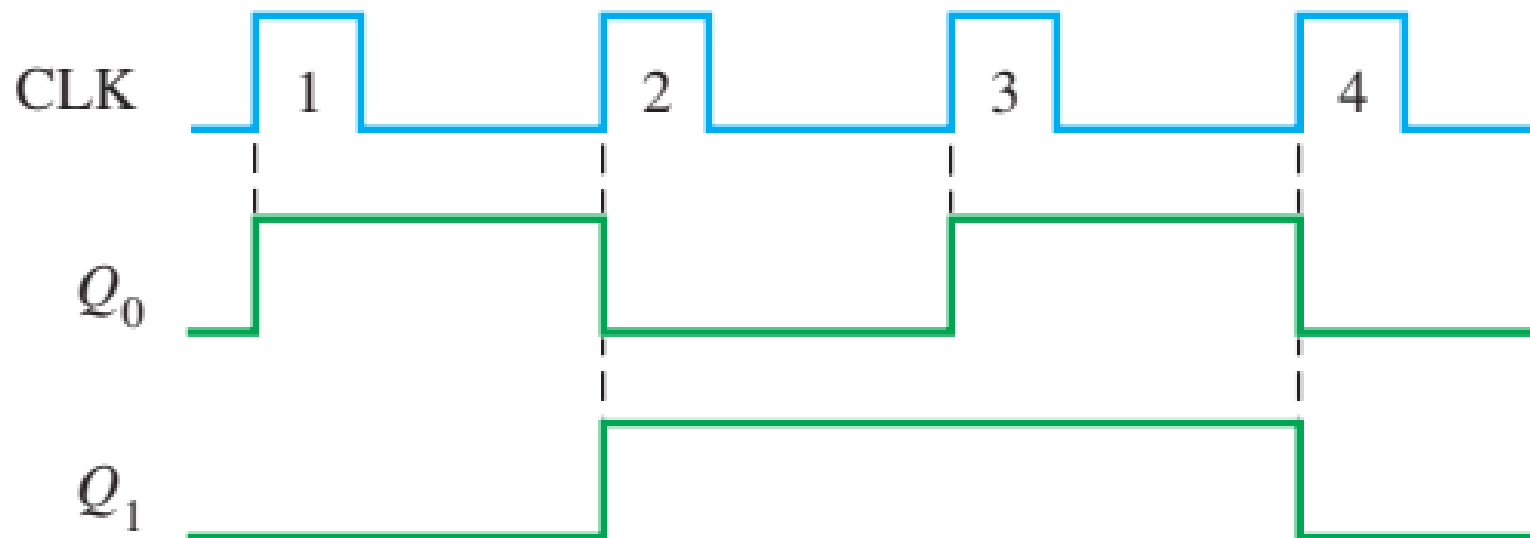
Two bit Synchronous Counter

- Assume that the counter is initially in the binary 0 state, that is, both flip-flops are **RESET**.
- During positive edge of **CLK1** pulse **FF0** will toggle and **Q_0** will go **high**. Due to the propagation delay **$J_1 = K_1 = 0$** therefore **FF1** does not change state.
- When positive edge of **CLK2** occurs, **FF0** will toggle and **Q_0** will go **low** and since **$J_1 = K_1 = 1$** then **FF1** will toggle and **Q_1** goes high.
- When the leading edge of **CLK3** occurs **FF0** toggles **Q_0** will go **high** and **Q_1** remains **high**.
- Finally at the leading edge of **CLK4** both **Q_0** and **Q_1** go low because they both have a toggle condition.

Synchronous(Parallel) Counters

Two bit Synchronous Counter

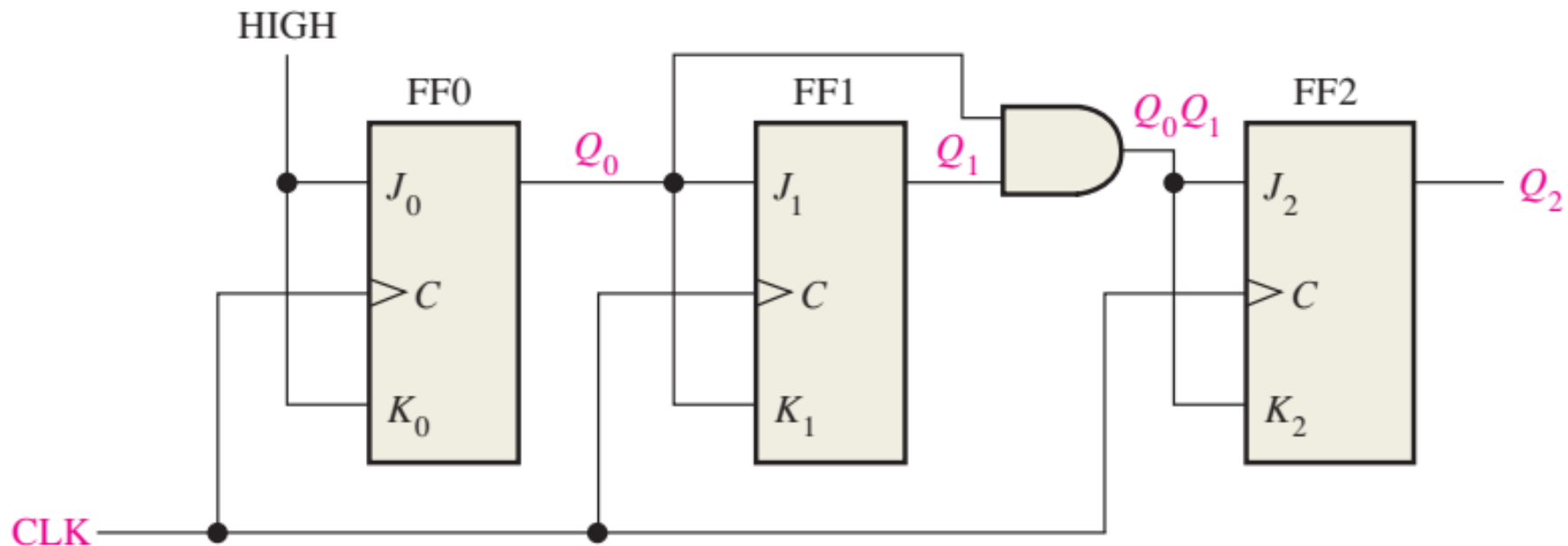
- The complete timing diagram for this counter is shown in figure



Synchronous(Parallel) Counters

A 3-Bit Synchronous Binary Counter

- A three bit synchronous counter and its binary state sequence are shown below



Synchronous(Parallel) Counters

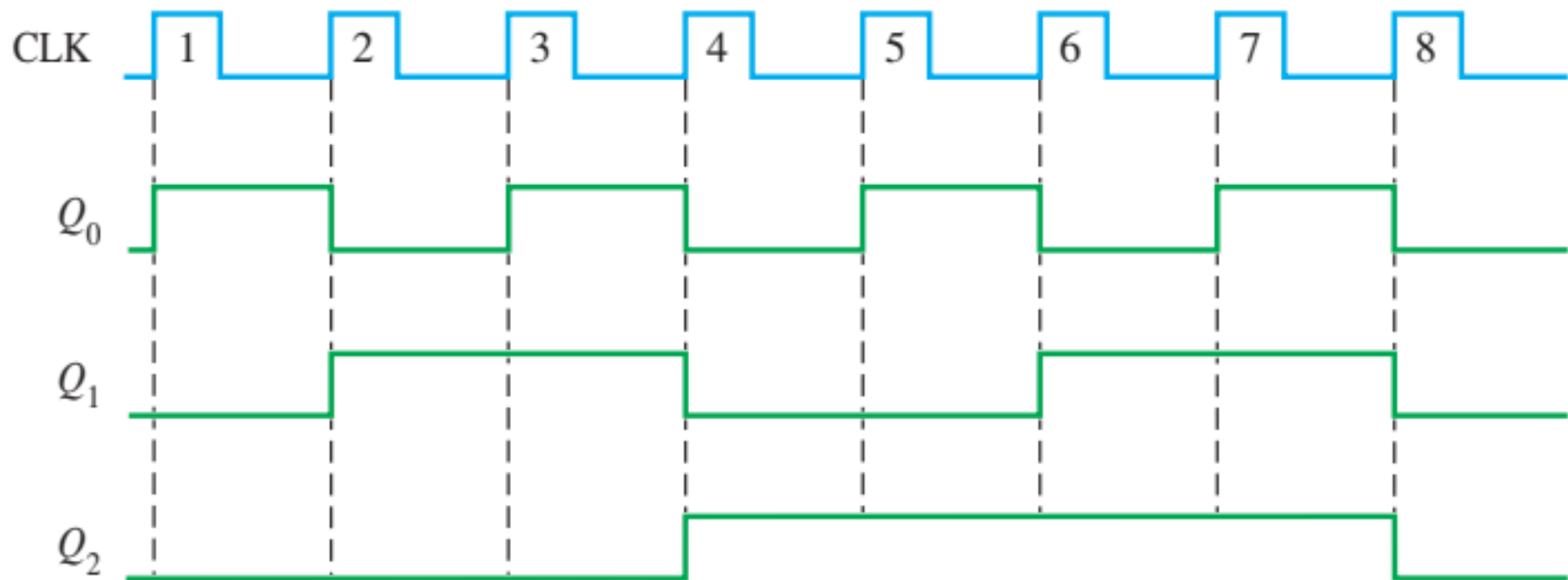
Three bit Synchronous Counter

- **FF0** must be held in **toggle** mode **Q_0** changes on each clock pulse.
- **Q_0** is connected to the **J_1** and **K_1** inputs of **FF1** hence **Q_1** changes each time **Q_0** is high and remains on its previous state when **Q_0** is **low**.
- **Q_2** changes state when it is preceded by the condition in which both **Q_0** and **Q_1** are **high**.
- This condition is detected by **AND gate** and applied to the **J_2** and **K_2** inputs of **FF2**. at all times the **J_2** and **K_2** inputs of **FF2** are held **low** by the **AND gate** output and **FF2** does not change state.

Synchronous(Parallel) Counters

Three bit Synchronous Counter

➤ The timing diagram of 3 bit counter is shown below



Synchronous(Parallel) Counters

Three bit Synchronous Counter

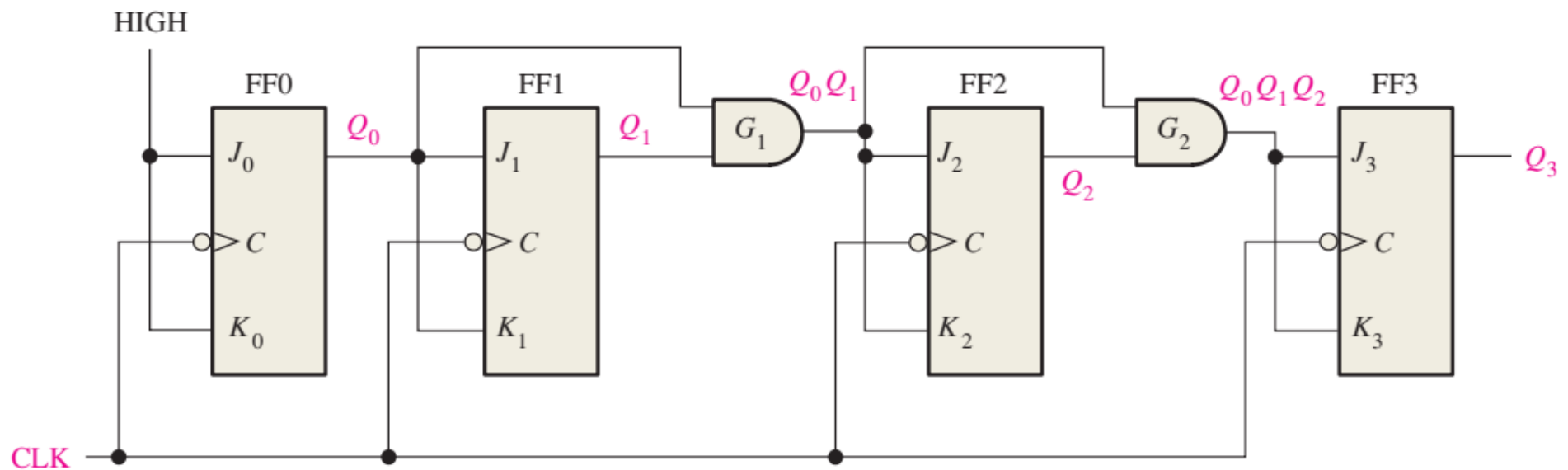
➤ State sequence for a 3-bit binary counter is shown below

Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

Synchronous(Parallel) Counters

A 4-Bit Synchronous Binary Counter

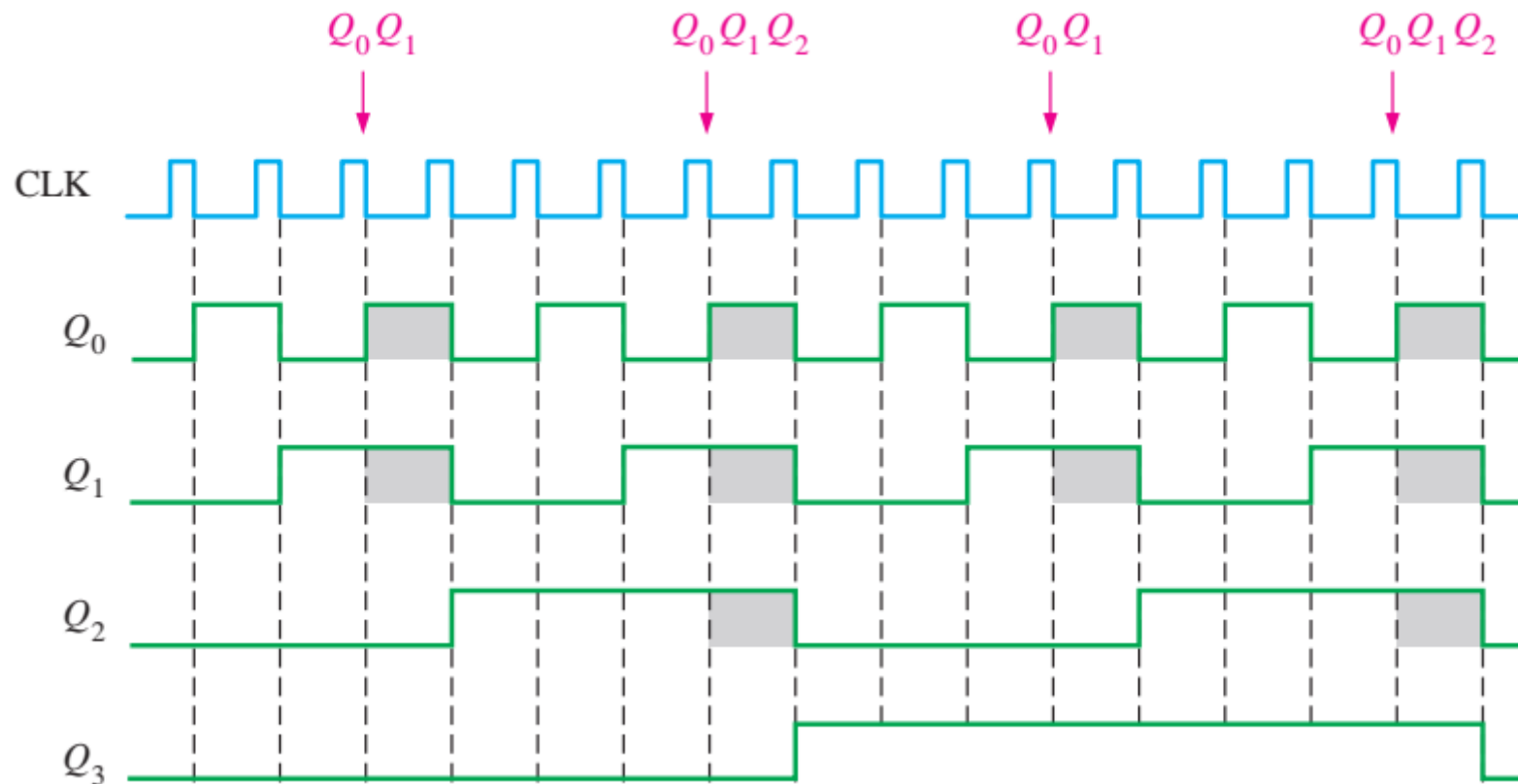
➤ A Four bit synchronous counter and its binary state sequence are shown below



Synchronous(Parallel) Counters

A 4-Bit Synchronous Binary Counter

➤ A Four bit synchronous counter



Design of Synchronous Counters

- Sequential circuits can be classified into **two types**:
 - ✓ Those in which the **output or outputs depend** only on the present **internal state** (**Moore state machines**)
 - ✓ Those in which the **output or outputs depend** on both the **present state** and **the input or inputs** (**Mealy state machines**).
- Besides the binary sequence, synchronous counters can be design to output any **random sequence**.
- We can **design** a counter which sequence only even numbers or odd numbers or gray code sequence etc.

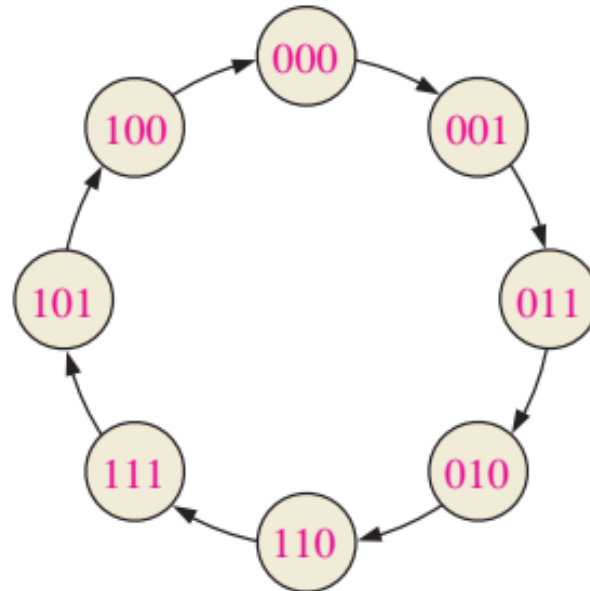
Design of Synchronous Counters

- Sequential circuits can be classified into **two types**:
 - ✓ Those in which the **output or outputs depend** only on the present **internal state** (**Moore state machines**)
 - ✓ Those in which the **output or outputs depend** on both the **present state** and **the input or inputs** (**Mealy state machines**).
- Besides the binary sequence, synchronous counters can be design to output any **random sequence**.
- We can design a counter which sequence only even numbers or odd numbers or gray code sequence etc..
- In this part the general design procedure for synchronous counter is applied in series of steps.

Design of Synchronous Counters

Step 1: State Diagram

- The first step in the design of a **state machine (counter)** is to create a state diagram.
- A state diagram shows the progression of states through which the counter advances when it is clocked.



Design of Synchronous Counters

Step 2: Next-State Table

- Derive a next-state table from the state diagram.
- It lists each state of the counter (**present state**) along with the corresponding **next state**.
- The next state is the state that the counter goes to from its present state upon application of a **clock pulse**.
- The next-state table is derived from the state diagram

Design of Synchronous Counters

Step 2: Next-State Table

Present State			Next State		
Q_2	Q_1	Q_0	Q_2	Q_1	Q_0
0	0	0	0	0	1
0	0	1	0	1	1
0	1	1	0	1	0
0	1	0	1	1	0
1	1	0	1	1	1
1	1	1	1	0	1
1	0	1	1	0	0
1	0	0	0	0	0

Design of Synchronous Counters

Step 3: Flip-Flop Transition Table

- All possible output transitions are listed by showing the **Q output** of the flip-flop going from **present states** to **next states**.
- Q_N is the **present state** of the flip-flop (**before a clock pulse**) and Q_{N+1} is the **next state** (after a clock pulse).
- For each output transition, the J and K inputs that will cause the transition to occur are listed.
- An **X** indicates a “**don't care**” (the input can be either a 1 or a 0).

Design of Synchronous Counters

Step 3: Flip-Flop Transition Table

Output Transitions			Flip-Flop Inputs	
Q_N		Q_{N+1}	J	K
0	→	0	0	X
0	→	1	1	X
1	→	0	X	1
1	→	1	X	0

Q_N : present state

Q_{N+1} : next state

X: “don’t care”

Design of Synchronous Counters

Step 3: Flip-Flop Transition Table

- The first row shows that to let the **present state 0** remain a **0 after the next clock pulse** then $J=0$ and K is don't care.
- $J=K=0$ relates to no change input condition and when $J=0, K=1$ relates to the reset input condition. Both of these cases results on the next state 0, therefore transition 0 to 0 is defined by $J=0, K=x$.
- All other rows can be similarly verified.

Design of Synchronous Counters

Step 3: Flip-Flop Transition Table

➤ Excitation table can be summarized as follow

Cell #	Present State			Next State			Flip flop inputs					
	Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	0	1	0	X	0	X	1	X
1	0	0	1	0	1	1	0	X	1	X	X	0
2	0	1	0	1	1	0	1	X	X	0	0	X
3	0	1	1	0	1	0	0	X	X	0	X	1
4	1	0	0	0	0	0	X	1	0	X	0	X
5	1	0	1	1	0	0	X	0	0	X	X	1
6	1	1	0	1	1	1	X	0	x	0	1	X
7	1	1	1	1	0	1	X	0	X	1	X	0

Design of Synchronous Counters

Step 4: Karnaugh Maps

- Karnaugh maps can be used to determine the **logic** required for the **J** and **K** inputs of each flip-flop in the counter.
- Transfer the J and K states from the transition table to the Karnaugh maps. There is a Karnaugh map for each input of each flip-flop.

Design of Synchronous Counters

Step 4: Karnaugh Maps

Q_2Q_1	Q_0	
	0	1
00	0	0
01	1	0
11	X	X
10	X	X

J_2 map

$Q_1\bar{Q}_0$

Q_2Q_1	Q_0	
	0	1
00	0	1
01	X	X
11	X	X
10	0	0

J_1 map

$\bar{Q}_2Q_0$

Q_2Q_1	Q_0	
	0	1
00	1	X
01	0	X
11	1	X
10	0	X

J_0 map

$\bar{Q}_2\bar{Q}_1$

Q_2Q_1

Q_2Q_1	Q_0	
	0	1
00	X	X
01	X	X
11	0	0
10	1	0

K_2 map

$\bar{Q}_1\bar{Q}_0$

Q_2Q_1	Q_0	
	0	1
00	X	X
01	0	0
11	0	1
10	X	X

K_1 map

Q_2Q_0

Q_2Q_1	Q_0	
	0	1
00	X	0
01	X	1
11	X	
10	X	1

K_0 map

$\bar{Q}_2Q_1$

$Q_2\bar{Q}_1$

Design of Synchronous Counters

Step 5: Logic Expression for Flip-Flop Inputs

- From the Karnaugh maps you can obtain the following expressions for the J and K inputs of each flip-flop and derive the logic expression for each flip-flop.

$$J_0 = Q_2Q_1 + \overline{Q_2}\overline{Q_1} = \overline{Q_2 \oplus Q_1}$$

$$K_0 = Q_2\overline{Q_1} + \overline{Q_2}Q_1 = Q_2 \oplus Q_1$$

$$J_1 = \overline{Q_2}Q_0$$

$$K_1 = Q_2Q_0$$

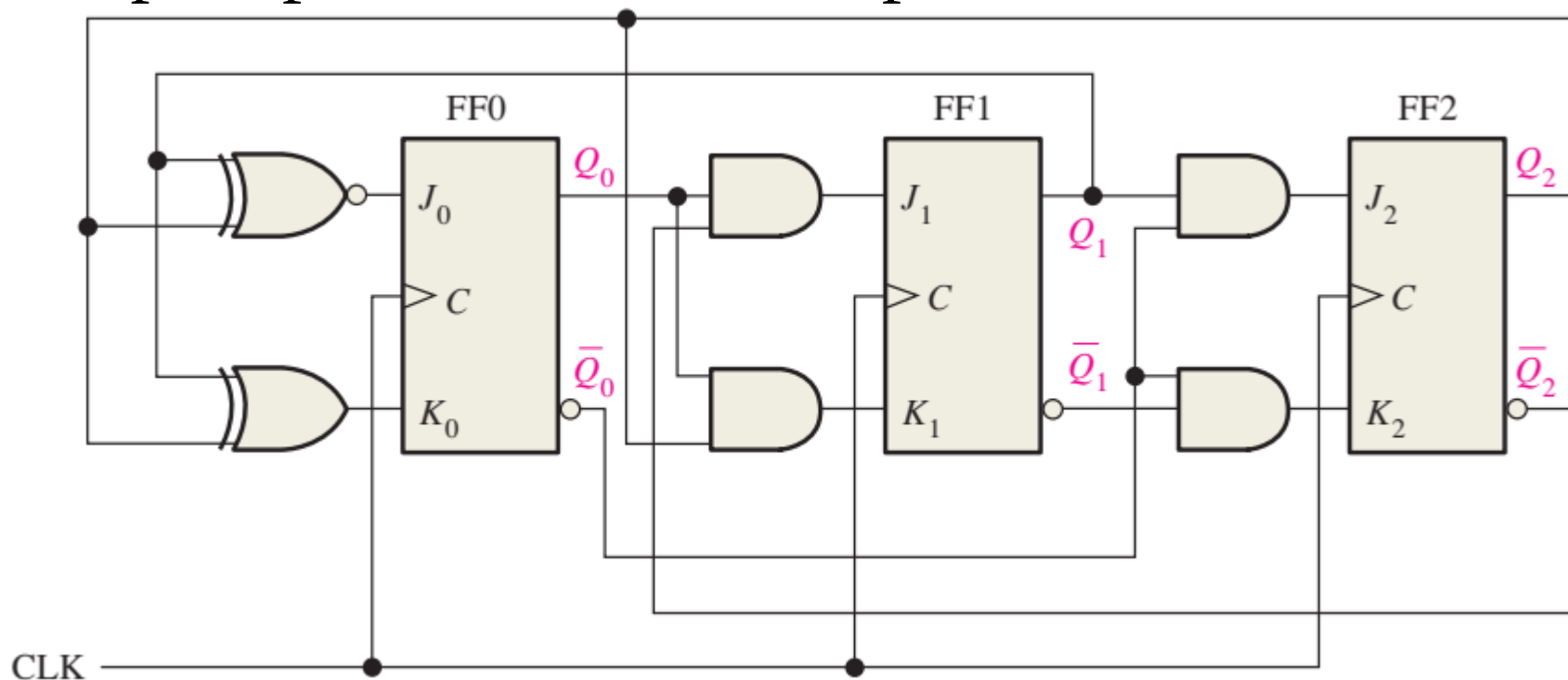
$$J_2 = Q_1\overline{Q_0}$$

$$K_2 = \overline{Q_1}\overline{Q_0}$$

Design of Synchronous Counters

Step 6: Counter Implementation

➤ The **final step** is to implement the combinational logic from the expressions for the **J** and **K** inputs and connect the flip-flops to form the complete counter.



Design of Synchronous Counters

In Summary

- In general, these steps can be applied to any state machine.
 1. Specify the counter sequence and draw a state diagram.
 2. Derive a next-state table from the state diagram.
 3. Develop a transition table showing the flip-flop inputs required for each transition. The transition table is always the same for a given type of flip-flop.
 4. Transfer the J and K states from the transition table to Karnaugh maps. There is a Karnaugh map for each input of each flip-flop.

Design of Synchronous Counters

In Summary

- In general, these steps can be applied to any state machine.
 5. Group the Karnaugh map cells to generate and derive the logic expression for each flip-flop input.
 6. Implement the expressions with combinational logic, and combine with the flip-flops to create the counter.

Tutorial

- The content of a 4-bit register is initially 1101. The register is shifted six times to the right with the serial input being 101101. What is the content of the register after each shift?
- Design a synchronous counter to sequence $4 \rightarrow 3 \rightarrow 1 \rightarrow 7 \rightarrow 5$ and repeat.
- Design a 3-bit binary ripple up counter using positive triggered D flip-flops.

Tutorial

➤ Design a 3-bit counter which counts in the sequence: 001, 011, 010, 110, 111, 101, 100, (repeat) 001, . . .

- a) Use J-K flip-flops
- b) Use S-R flip-flops
- c) Use D flip-flops

Note: Transition Tables D and SR flip-flops are

Row #	Q	Q^+	D
0	0	0	0
1	0	1	1
2	1	0	0
3	1	1	1

Row #	Q	Q^+	S	R
0	0	0	0	X
1	0	1	1	0
2	1	0	0	1
3	1	1	X	0

Homework

Study on

- Synchronous updown counter.
- Cascaded Counters.
- Counter Decoding
- Finite State Machines
- Counter Applications

Homework

- Read Chapter 9 of Digital Fundamentals by Floyd and do Problems found in page no 550 -560.

**NEXT LECTURE
TEST 1**